

A Signal in the Noise: Using Machine Learning to Improve TESS's Systematic Error and Find More Planets

A thesis presented

by

Arielle C. Frommer

to

The Department of Astronomy

in partial fulfillment of the requirements

for the degree of

Bachelor of Arts

in the subject of

Astrophysics & Physics

Harvard University

Cambridge, Massachusetts

April 2025

© 2025 — Arielle C. Frommer

All rights reserved.

A Signal in the Noise: Using Machine Learning to Improve TESS's Systematic Error and Find More Planets

Abstract

The Transiting Exoplanet Satellite Survey (TESS) searches for exoplanets via the transit method, collecting continuous light curves for about 30 days at a time. TESS currently applies the Pre-search Data Conditioning (PDC) noise correction algorithm to correct for systematic error originating from known and unknown causes. While PDC uses linear combinations of feature vectors to extract systematic noise from the data, other machine learning techniques could offer more sophisticated means of systematic noise correction. We investigate the possibility of using a fully-connected autoencoder, which can similarly extract the key features of the light curves, to perform these systematic noise corrections. The instrumental systematics are preserved in the autoencoder reconstruction, while transits unique to each light curve are ignored. We can therefore subtract the reconstructions from the raw light curves to produce a noise-corrected light curve. After training and testing our autoencoder on the light curves, we found that our pipeline removes 0.3% more noise than PDC. However, the resulting light curves had modified or damped transits, indicating that the autoencoder is learning to reconstruct specific astrophysical signals in addition to systematics. Constraining the autoencoder so that it learns systematic noise features but ignores individual planetary signals is essential to building an effective noise correction pipeline. Ultimately, if our autoencoder improves the systematic error floor while preserving transits, it will enable the detection of more planets and improve the measurement precision of existing planets.

Contents

Abstract	iii
Acknowledgments	vii
Dedication	ix
1 Introduction	1
1.1 A Brief History of Exoplanet Detection	1
1.2 The Transiting Exoplanet Satellite Survey	4
1.2.1 SPOC Pipeline	6
1.2.2 Pre-search Data Conditioning	6
1.2.3 Motivations for Noise Correction	8
1.3 Machine Learning in Astronomy	11
1.3.1 Neural Networks and Autoencoders	12
1.3.2 Past Applications to Exoplanet Science	17
1.3.3 Noise Reduction Pipeline	19
1.4 Scientific Goals	20
1.5 Outline	21
2 Methods	22
2.1 Preprocessing Pipeline	22
2.1.1 Sector Selection	22

CONTENTS

2.1.2	TESS Quality Flags	23
2.1.3	NaN Preprocessing	24
2.1.4	Sigma-Clipping and Normalizing	25
2.2	Constructing the Autoencoder	26
2.3	Training	27
2.3.1	Binning	27
2.3.2	Random Search	28
2.4	Reconstruction and Subtraction	30
2.4.1	Postprocessing	31
3	Results	34
3.1	Subtractions	34
3.2	Comparing RMS	35
3.3	Comparing TOIs	37
3.3.1	Known Planets	37
3.3.2	Sigma-Clipping	37
3.3.3	Confirmed Planets	41
4	Discussion	45
4.1	Improving Our Pipeline	45
4.1.1	Preserving Transit Signals	45
4.2	Future Steps	47
4.2.1	Preprocessing	47
4.2.2	Training	48
4.2.3	Comparing Pipelines	49
4.2.4	Searching for Transits	51
4.2.5	Removing Noise From Subsequent Sectors	51
4.3	Conclusions	51

CONTENTS

References

53

Acknowledgments

I would like to sincerely thank my advisor, Dr. Jason Eastman, for being a wonderful mentor to me over the last two years. He has always been a fantastic advisor, from supporting our exciting junior thesis project where we discovered and modeled a complex triple star system to encouraging me to pursue my interest in machine learning for my senior thesis. I always appreciate his willingness to answer my questions and support my research interests in whatever directions they may take me. My first project with him ignited my passion for exoplanets research, and I am so grateful for him for supporting that spark, helping me grow as a researcher and scientist, and advising my thesis research throughout this year and beyond.

I also would like to thank Professor Andrew Vanderburg and Dr. Chris Shallue for their advice on this thesis. They helped generate the seed for this project over the summer, and I am so grateful to them for their guidance and enthusiasm for the project. In particular, I am grateful to Dr. Shallue for mentoring me throughout the thesis and advising me on the machine learning side of things.

I would like to thank Professor Hernquist and my classmates in Astronomy 99 for a wonderful senior thesis experience. I would also like to thank my friends in astronomy – Rafid Quayum, Stephanie Yoshida, Ellie Griffin – for supporting me and being there since day one.

To my parents and my brothers, Gaby, Sammy, and Rafi, thank you for supporting my love for space and inspiring me to dream big. I would like to thank my dad for giving me books about Hubble and teaching me about space since I was a toddler. I would like

CHAPTER 0. ACKNOWLEDGMENTS

to thank my mom for all of her unconditional love and enthusiasm, and for lots of long phone calls these last few weeks. I would like to also thank my grandpa, Papa Peter, for encouraging me to always be curious about the world.

This thesis project has been a culmination of my academic and research experiences at Harvard. I am so grateful for my friends, mentors, and professors along the way who have supported me and my dreams. I am so lucky to have so many wonderful people around me, and I would like to thank them – in particular, Uzma Issa, Alison Chan, Imaan Mirza, Abby Miller, Krisha Patel, and Zakarias Erdos – for being an endless source of laughter and support this year.

To my parents, Laura and Ian, for always believing in me and supporting my dreams.

Chapter 1

Introduction

1.1 A Brief History of Exoplanet Detection

For centuries, astronomers have speculated about the existence of planets beyond our solar system. The first indirect evidence for a planet's existence emerged in 1917, when Walter Sydney Adams observed the spectrum of a white dwarf star, revealing the presence of heavy elements in its atmosphere (see Figure [1.1](#)). While not recognized at the time, astronomers realized nearly 90 years later that the source of the elements came from the debris of an exoplanet that had been engulfed by its star ([Zuckerman 2014](#)).

In the 1950s, scientists had proposed using Doppler spectroscopy and transits to search for planetary signals, techniques which we use today to detect them ([Struve 1952](#)). In 1963, Peter van de Kamp claimed to have discovered a Jupiter-sized planet from the wobbles of Barnard's star ([van de Kamp 1969](#)). Although van de Kamp's claim was later attributed to telescope errors, modern astronomers revisited Barnard's star using more

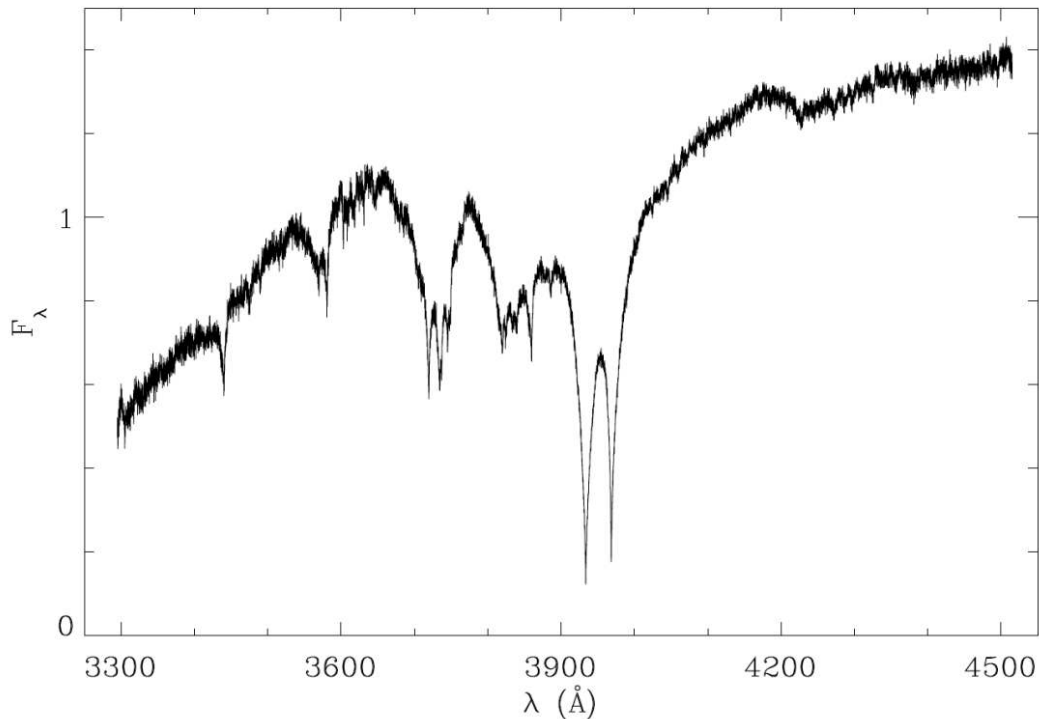


Figure 1.1: The first indirect evidence for an exoplanet came from the spectrum of van Maanen’s star, which had metallic lines of iron, calcium, and magnesium now thought to originate from a planet that fell into the star. Source: [Zuckerman \(2014\)](#).

precise radial velocity techniques, confirming the presence of four tiny planets orbiting it ([Basant et al. 2025](#)).

The first true exoplanet detections began in the late 1990s. In 1989, two astronomers published radial velocity observations of potential planetary candidates. The first candidate, Tadmor, was initially dismissed as noise from stellar rotations but was later confirmed as an exoplanet in 2002 ([Cochran et al. 2002](#)). The second was HD 114762 b, which was initially identified as a brown dwarf — an object that hovers between planet and star, unable to fuse hydrogen but retaining some stellar properties ([Latham et al.](#)

CHAPTER 1. INTRODUCTION

1989). The object – discovered by Harvard’s Dave Latham – was subsequently included in planet catalogs until it was finally ruled out as a planet in 2019 (Kiefer 2019).

The first confirmed detection of an exoplanetary system came in 1992, when Aleksander Wolszczan discovered a system of terrestrial-mass planets orbiting a millisecond pulsar (Wolszczan & Frail 1992). The first detection of an exoplanet around a main-sequence star like our sun came in 1995, when Michel Mayor and Didier Queloz used radial velocity measurements to identify a Jupiter-mass companion (Mayor & Queloz 1995).

Now, thousands of planets have been discovered using radial velocity, transits, direct imaging, microlensing, and more exotic techniques. As of this paper, 5,862 planets outside our solar system have been confirmed to exist, according to the NASA Exoplanet Archive. Astronomers use powerful instruments such as the Hubble Space Telescope, Kepler Space Telescope, Transiting Exoplanet Survey Satellite (TESS), and now the James Webb Space Telescope (JWST) to discover and characterize these alien worlds.

This rich history of exoplanet detection has featured uncertainty, disputed findings, and retracted claims along the way, and astronomers have faced countless uncertainties and false leads in the search for exoplanets. Yet through increasingly precise methods, they have uncovered thousands of distant worlds. By developing a new technique to reduce noise in exoplanet light curves, this thesis aims to build on that legacy – continuing the search for a signal in the noise.

1.2 The Transiting Exoplanet Satellite Survey

Today, most exoplanets are detected using the transit method. As a planet passes in front of its star, the star’s light will dim periodically, producing a small dip in the stellar light curve. A planet’s transit provides key information about its size, orbit, and alignment relative to Earth. The depth of the transit reveals the ratio of the planet’s radius to that of its star. The transit duration, along with the ingress and egress – the moments when the planet begins and ends its transit – depend on the planet’s orbital alignment relative to Earth. For example, planets whose orbits are highly inclined relative to Earth will have shorter transit durations.

The first confirmed exoplanet transit was observed in 1999 by two teams, including Harvard’s own David Charbonneau ([Charbonneau et al. 2000](#); [Henry et al. 2000](#)). Since then, the transit method has detected thousands of exoplanets through large sky surveys. In particular, the Transiting Exoplanet Satellite Survey (TESS) was launched in 2018 and designed to survey the entire sky in 2 years ([Ricker et al. 2014](#)). TESS observes the sky in sectors, monitoring each patch for approximately 27 days to detect periodic dips in stellar brightness.

Figure [1.2](#) demonstrates the span that TESS can cover in 2 years. Depending on a star’s ecliptic latitude, that star could be for a period ranging from 27 days to 351 days, if the sectors overlap. The survey was designed so that the longest observing intervals were near the ecliptic poles, which are the best locations for follow-up observations with JWST.

The length of sector observations was determined based on the planetary periods

CHAPTER 1. INTRODUCTION

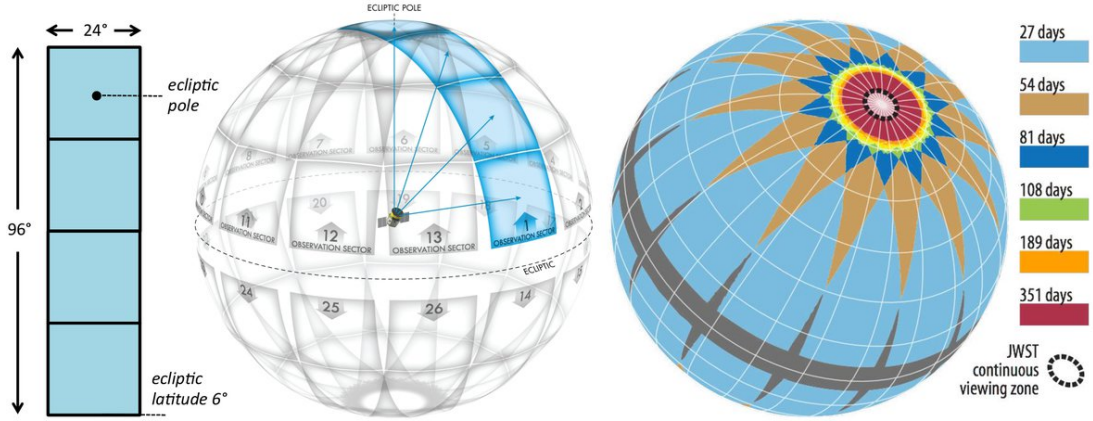


Figure 1.2: TESS scans the sky for exoplanets in sectors, covering the entire sky every 2 years. Source: [Ricker et al. \(2014\)](#). The survey is designed to observe planets near the ecliptic poles for longer periods of time based on the overlap of sectors, since those stars are at optimal location for follow-up study with JWST. The dotted line indicates the region in which JWST will be able to take observations at any time.

TESS aimed to target. Targeting longer-period planets can find more potentially habitable worlds, but also increases mission cost. The maximum periodicity of planets discovered by Kepler was around 10 days, so the TESS survey set a 27-day sector length to target planets with a maximum period of around 10 days. Planets with up to 40-day periods could be detected closer to the ecliptic poles.

Among other targets, TESS can detect small, Earth-sized planets around nearby bright stars and M dwarfs, advancing the search for potentially habitable terrestrial worlds – although TESS is also biased to short-period planets that are less likely to be habitable. However, these discoveries nonetheless can provide prime candidates for follow-up spectroscopic observations to measure their masses and characterize their atmospheres.

1.2.1 SPOC Pipeline

The raw data from TESS is processed through the Science Processing Operations Center (SPOC) pipeline, which generates light curves and associated datasets for the scientific community. When TESS detects periodic flux dimmings in a star, SPOC classifies the event as a Threshold Crossing Event (TCE), using both short-cadence (2-minute) and long-cadence (30-minute) observations, which are then sent to the TESS Science Office (TSO).

The TSO then analyzes all TCEs to filter out non-planetary signals, such as eclipsing stellar binaries, variable stars, or instrumental systematics. The remaining candidates are labeled Targets of Interest (TOIs) and released publicly for follow-up observations. As of today, TESS has identified 7,576 planetary candidates, of which 620 have been confirmed as exoplanets, making up around 10% of all confirmed planets (by comparison, Kepler and its extended mission K2 confirmed around 56% of all known planets).

1.2.2 Pre-search Data Conditioning

However, a major limiting factor on the precision of TESS data is systematic noise, which affects entire observational sectors. Systematic noise arises from astrophysical signals or known and unknown issues with the telescope, such as pointing drift, focus changes, thermal gradients, scattered light, and velocity aberrations. Since systematics can obscure and bias true planetary signals, correcting for them is not only beneficial but often necessary.

To mitigate these issues, TESS applies a noise correction algorithm known as

CHAPTER 1. INTRODUCTION

Pre-search Data Conditioning (PDC). First developed for the Kepler mission ([Smith et al. 2012](#)), PDC aims to remove systematic trends caused by the telescope while preserving individual transits and intrinsic stellar variability. The goal of PDC is to remove instrumental systematics while preserving astrophysical signals. The pipeline exploits the fact that instrumental signals should be highly correlated across targets, while stellar variability and unique noise features are not.

PDC operates by analyzing the quietest light curves in a given dataset to identify the systematics that appear across multiple stars. These trends are stored as Cotrending Basis Vectors (CBVs) and ranked according to how strongly they contribute to systematic noise across an entire detector channel (see [Figure 1.3](#)).

The CBVs are subtracted from each light curve using tunable weights that optimize the fit. PDC builds on a maximum likelihood approach, modeling each light curve as a linear combination of its CBVs and solving for the weights using Bayesian priors to find the combination that matches the light curve best, without overfitting. The CBVs are generated by singular value decomposition (SVD), a matrix factorization technique that can be used to extract structure from data. Using linear algebra, SVD identifies systematic trends from the light curves by capturing the strongest correlated patterns.

A key challenge is balancing noise removal with preserving genuine astrophysical signals like stellar variability or other unique data features that scientists are interested in. Underfitting can leave systematic errors that can obscure exoplanet transits, while overfitting risks erasing real periodic signals or stellar variability. An effective systematic noise correction ensures that TESS light curves eliminate noise artifacts that can obscure signals while maintaining the signals themselves.

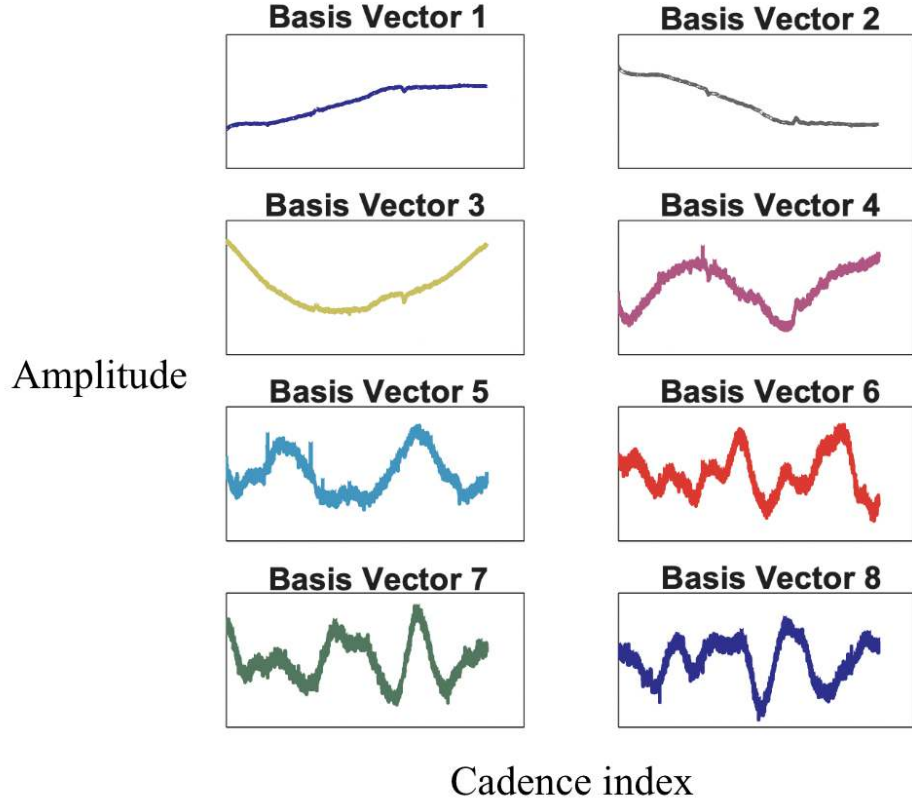


Figure 1.3: Pictured above are 8 Cotrending Basis Vectors in Kepler channel 2.1. The x-axis represents the time, or cadence index, while the y-axis represents the amplitude of the noise features. Source: [Smith et al. \(2012\)](#).

1.2.3 Motivations for Noise Correction

Why strive for better noise subtraction? The majority of TESS TOIs have low signal-to-noise ratios, which is calculated by finding the root mean square (RMS) noise of the data:

$$\text{RMS} = \sqrt{\frac{\sum^n (x_i)^2}{n}} \quad (1.1)$$

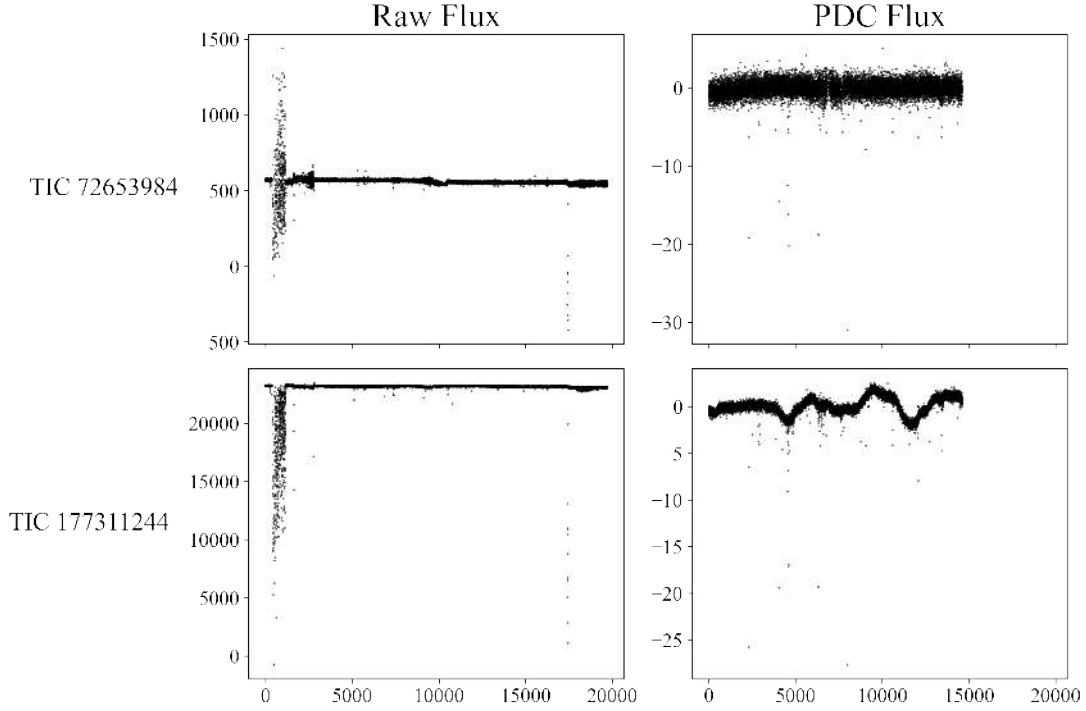


Figure 1.4: Example of raw light curves before and after PDC, which removes systematic noise in the light curves.

where x_i is the dataset and n is the number of data points – effectively calculating the mean of the dataset squared. RMS quantifies the scatter of the data, which is inversely related to its signal-to-noise (SNR) ratio. SNR quantifies the significance with which you’ve detected a planet, and is dependent on RMS, the depth of the light curve, and other features. [Carter et al. \(2008\)](#) defines the SNR of a light curve:

$$\text{SNR} = \sqrt{\Gamma T} \frac{\delta}{\text{RMS}} \quad (1.2)$$

where Γ is the rate at which data points are sampled, T is the duration of the transit, and δ is the transit depth. Decreasing the scatter or RMS of a light curve by removing instrumental systematics increases SNR, making it easier to detect planetary

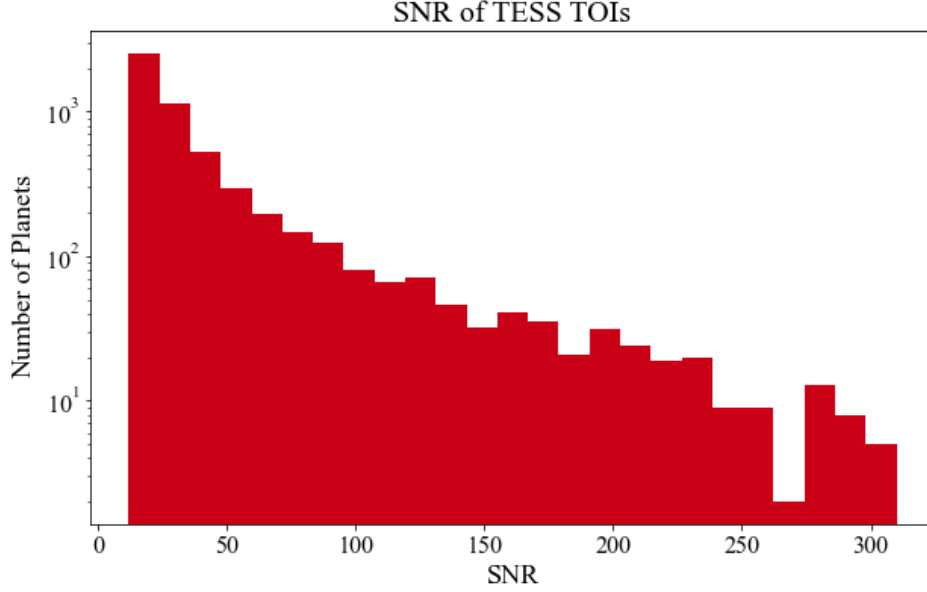


Figure 1.5: The frequency of TOIs increases as SNR decreases, emphasizing the importance of improving noise correction to access planets in this noisier regime.

transits.

Figure 1.5 shows a log-scaled histogram of TESS TOIs by SNR. The number of planetary candidates increases exponentially as SNR decreases. For example, improving SNR by 1% would add over 250 TOIs that previously had SNR too low to detect a signal through. Effective noise subtraction is therefore essential for detecting and characterizing these planets, as it can improve their SNR. It is often the most interesting planets — those with the longest orbit periods, most active stellar hosts, or smallest radii — that are hardest to detect because of low SNR. Pursuing ways to correct for systematic noise across all these targets has tremendous potential to improve the precision of low SNR discoveries, and find new candidates among currently insignificant signals, increasing our sensitivity to low-mass, more Earth-like planets.

Currently, PDC noise correction relies on a linear algorithm, subtracting Cotrending Basis Vectors (CBVs) using linear weights. However, nonlinear machine learning techniques that have greater flexibility could offer more sophisticated means of systematic noise correction than PDC. Developing and testing such an algorithm is the central goal of this thesis.

1.3 Machine Learning in Astronomy

Machine learning is a branch of data science where computer algorithms learn patterns and features in data without explicit programming. Rooted in statistics and optimization, machine learning employs various models and architectures to predict trends, classify data, or generate new information. With its computational power and ability to recognize complex patterns, machine learning has become a versatile and indispensable tool across multiple disciplines, including the natural sciences.

Astronomy, in particular, is a highly data-driven field. Large sky surveys such as the Sloan Digital Sky Survey (SDSS), which identifies millions of galaxies, stars, and quasars at different redshifts, or Gaia, which precisely measures the distances, positions, and motions of billions of stars, have generated vast repositories of data. These datasets provide rich opportunities for machine learning applications, including classification, anomaly detection, clustering, noise removal, and computational modeling ([Baron 2019](#)).

Machine learning approaches can be broadly categorized into supervised and unsupervised learning. Supervised learning involves training an algorithm on labeled data, allowing it to recognize specific categories – for example, training a model to

recognize different types of variable stars (Mahabal et al. 2017). In contrast, unsupervised learning requires the algorithm to recognize patterns and structures in data on its own. Unsupervised learning is used for various modes of data exploration, such as clustering, visualization, anomaly detection, and dimensionality reduction.

1.3.1 Neural Networks and Autoencoders

Neural networks are a class of machine learning algorithms inspired by biological neural processes in the brain. They consist of multiple layers of interconnected neurons, where each connection is assigned a weight. These weights are adjusted during training as the model learns to recognize patterns and features in the data.

The flexibility of neural networks comes from their construction – neurons of each layer are connected through nonlinear activation functions, such as a hyperbolic tangent function (1.4). These layers are known as "hidden layers," in which the neural network is learning how to map key inputs to an output. We calculate the value of each neuron by applying different weights and biases to x :

$$x' = \sum_i w_i x_i + b \quad (1.3)$$

where w_i are the individual weight parameters, x_i are the input features, and b is the bias term (set to zero in our particular model). For our model, we initialize the weights with a Kaiming normal distribution and set the initial bias to zero – the model will continuously update its weights and biases as it trains.

Each neuron is assigned a value based on the combined weights of previous neurons,

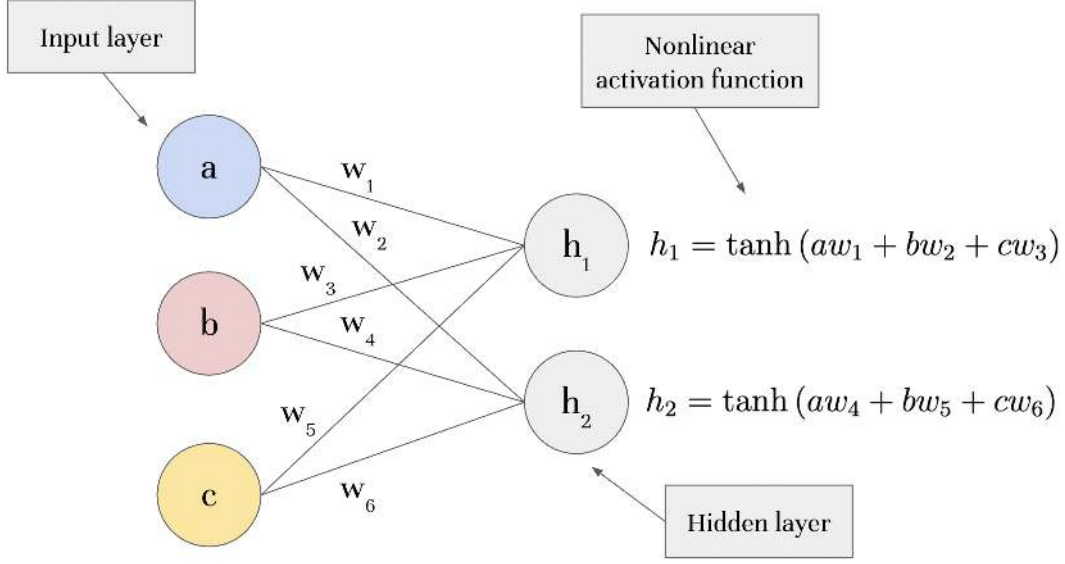


Figure 1.6: Each nonlinear layer passes a set of inputs and weights through an activation function, such as hyperbolic tangent. The autoencoder iteratively updates the weights and biases of each layer through backpropagation and gradient descent.

passed through a nonlinear activation function (see Figure (1.6)). Our model uses the hyperbolic tangent function, which is defined as:

$$f(x') = \tanh(x') = \frac{e^{x'} - e^{-x'}}{e^{x'} + e^{-x'}} \quad (1.4)$$

The hyperbolic tangent function is generally faster to train on because it is centered at 0, meaning it maps negative inputs to negative values and positive inputs to positive values, which helps the model learn more efficiently. Additionally, hyperbolic tangent functions work better for shallower networks with fewer layers.

When a neuron receives input from previous hidden layers, it computes a weighted sum of those inputs and applies the sigmoid function to produce an output:

$$h_1 = \tanh \left(\sum_i w_i x_i + b \right) \quad (1.5)$$

where x_i are the inputs from the previous layer, w_i are the corresponding weights, b is a bias term, and $\tanh$ is the nonlinear activation function. This nonlinearity allows neural networks to learn complex patterns more easily than linear optimization techniques, performing tasks like classification, dimensionality reduction, and feature extraction with incredible efficiency.

Neural networks can be supervised or unsupervised, and different neural network architectures can be used for various tasks. Convolutional neural networks (CNNs) and autoencoders have been used for different research applications in astronomy, though other types of neural networks are also utilized. CNNs are widely used for image and spatial data. They train through learning specific spatial or visual features, applying convolutional filters to capture local patterns, which makes them well-suited for image recognition or classification.

Autoencoders similarly aim to capture key features of the data. Autoencoders contain two parts – an encoder and decoder. Through a series of hidden layers, the encoder compresses the input data x into a latent space representation z , capturing its most significant features:

$$z = f_\theta(x) \quad (1.6)$$

where f_θ represents the encoding function with learnable parameters θ .

The decoder reconstructs the original input $\hat{x}$ from the latent space representation

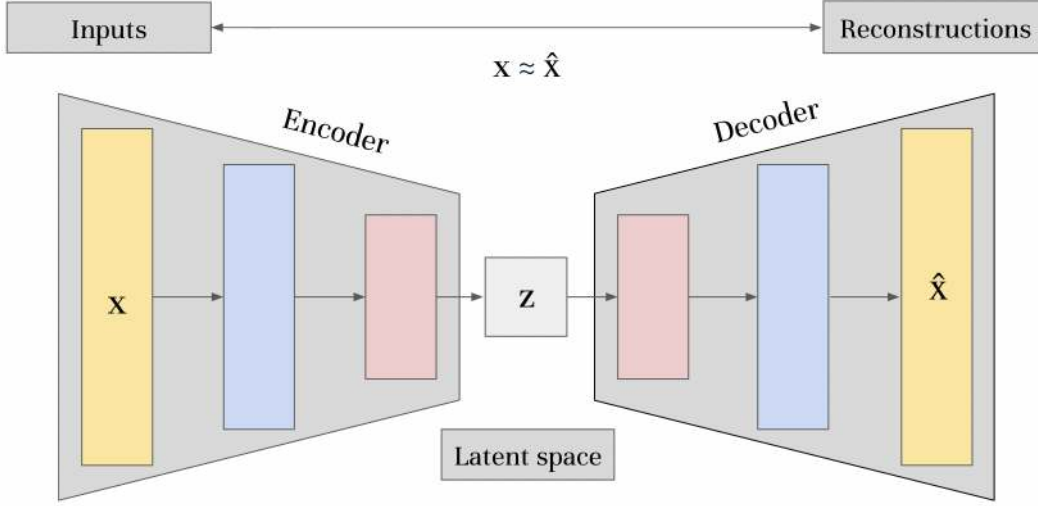


Figure 1.7: Autoencoders learn the common trends across a dataset by reducing the dimensionality through an encoder with hidden layers, reconstructing the inputs through a decoder, and aiming to minimize loss between the encoder and decoder.

z , through another set of nonlinear hidden layers whose weights are also optimized:

$$\hat{x} = g_{\phi}(z) \quad (1.7)$$

where g_{ϕ} represents the decoding function with learnable parameters ϕ .

The process of training an autoencoder involves minimizing a loss function, which quantifies the difference between the predicted and actual outputs. After passing its inputs through an encoder and decoder, the autoencoder uses the loss function to measure how well its reconstructed inputs match the original inputs. A common loss function is Mean Squared Error:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2 \quad (1.8)$$

which quantifies the difference between the reconstructions and the inputs. The optimization is performed using backpropagation, which works by computing the gradient of the loss function with respect to each weight in the network. Backpropagation updates weights iteratively through gradient descent by moving the weights in the direction that minimizes the loss. The update step is:

$$w_{i+1} = w_i - \eta \frac{\partial L}{\partial w_i} \quad (1.9)$$

The learning rate, η , sets how much the weights update with each iteration. Training the autoencoder to reconstruct the inputs well involves tuning the learning rate accordingly. If the learning rate is too large, the optimization may overshoot the weights that lead to minimum loss. If the learning rate is too small, convergence may be slow or the network may get stuck in local minima.

We can also add a momentum term to gradient descent to minimize loss faster:

$$w_{i+1} = w_i - \eta \frac{\partial L}{\partial w_i} + \gamma(w_i - w_{i-1}) \quad (1.10)$$

The momentum term includes the weight change from the previous iteration, multiplied by momentum factor. For example, if the previous weight change was large, then the momentum will help speed up the gradient descent.

Autoencoders are often used for dimensionality reduction, since they can extract the most common features across a dataset. A key element of the autoencoder is the

CHAPTER 1. INTRODUCTION

latent space, which enables it to learn systematic trends and patterns by imposing a forced bottleneck that reduces the dimensionality of each layer (see Figure 1.7). autoencoders are value tools for feature extraction, dimensionality reduction, and noise removal, as they can learn and separate systematic features from meaningful signals. More broadly, neural networks are aiding astronomers in classification, data analysis, and noise reduction (Smith & Geach 2023). As astronomical datasets continue to grow with large sky surveys, machine learning will become an increasingly valuable tool.

1.3.2 Past Applications to Exoplanet Science

Exoplanets is a heavily data-driven field, and machine learning has great potential to help astronomers analyze large datasets and classify exoplanet and stellar systems more effectively. In recent years, exoplanet scientists have utilized neural networks to perform tasks like classification, computation, noise removal, and more.

In Shallue & Vanderburg (2018), Chris Shallue and Andrew Vanderburg, who both advised this thesis project, developed a CNN to search for transits in noisy light curves from Kepler. They trained their neural network on a set of verified planets and strong candidates to help distinguish between false positives. Their neural network led to the discovery of two new planets, Kepler-90 i and Kepler-80 g. Later, Yu et al. (2019) modified this CNN to vet TESS candidates and identify planetary signals.

Machine learning has also been used for efficient data retrieval. Ardévol Martínez, F. et al. (2022) applied a CNN to retrieve complex models of exoplanet atmospheres, comparing its performance to traditional Bayesian retrieval techniques. Their results demonstrated that CNNs achieve comparable accuracy while requiring significantly less

CHAPTER 1. INTRODUCTION

computational effort.

Neural networks have also been proven effective at denoising data directly. [Morvan et al. \(2022\)](#) used another type of neural network known as a Transformer to remove noise in TESS light curves. The paper makes a case for the flexibility and increased performance that machine learning algorithms can bring to large datasets.

Machine learning has also been applied to radial velocity data; in [de Beurs et al. \(2022\)](#), astronomers trained an algorithm to predict and remove stellar activity from radial velocity measurements, helping to isolate planetary signals.

More recently, [Iglesias Álvarez et al. \(2023\)](#) trained a CNN to detect transits in simulated Kepler light curves, building an effective classifier that could identify a transit-like signal in a stellar light curve. In the field of exoplanet atmospheres, [Vasist et al. \(2023\)](#) used a neural posterior estimation to perform quicker atmospheric retrievals.

These applications highlight machine learning’s potential to improve detection sensitivity, refine planetary parameters, and reduce false positives. In many of these papers, machine learning is not a substitute for physics, but rather a valuable computational tool that can improve performance and learn the data better than more traditional techniques. This thesis aims to build on this precedent of using machine learning as an analytical tool. We are not aiming to use neural networks to predict new physics – rather, the autoencoder is a powerful computational tool that can efficiently extract and remove systematic noise features in the data.

1.3.3 Noise Reduction Pipeline

This thesis develops a machine learning pipeline using an autoencoder to correct for systematic noise in TESS light curves. Like PDC (see Section 1.2.2), the autoencoder extracts the key features of the light curves so that we can remove them and improve the signal-to-noise ratio.

Through dimensionality reduction and reconstruction, the autoencoder will learn the systematic noise features across the dataset while ignoring distinct transit signals. We will then subtract the autoencoder’s reconstructed light curves off of the original light curves, removing the systematic noise features while maintaining signals unique to each light curve.

The autoencoder technique differs from PDC in that it introduces a multi-layer, nonlinear, flexible network that allows the model to learn more complex relationships across the data, allowing us to remove the systematic noise while preserving the transits. The autoencoder is well-suited for this noise correction since it can learn systematic features that appear across all light curves – the structural noise – while ignoring the unique transit signals that we aim to reveal.

Figure 1.8 outlines the pipeline of our noise-correcting autoencoder. First, we preprocess the data by removing bad quality points and normalizing, which is further outlined in Section 2.1. Then, we train our autoencoder so that it can reconstruct the systematic noise of the light curves (see Section 2.3). After subtracting off the reconstructions from the normalized light curves and unnormalizing, we can compare our noise-corrected light curves to the PDC-corrected light curves (see Chapter 3).

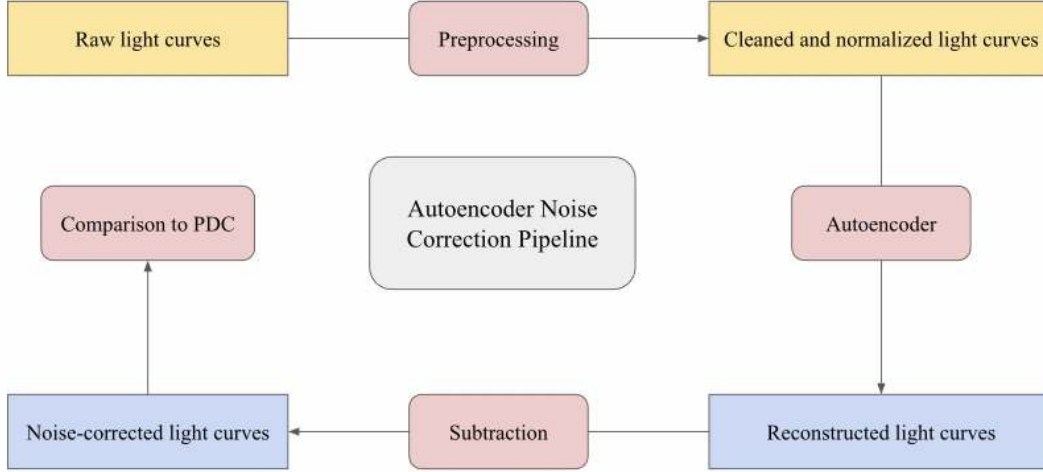


Figure 1.8: This figure illustrates the full autoencoder noise correction pipeline. Before feeding the light curves through the autoencoder, the light curves have to be appropriately preprocessed so that certain bad quality data points are removed and the light curves are normalized. Then, the light curves are passed through the Autoencoder, which reconstructs the systematic noise features. Finally, the reconstructions are subtracted off of the original data, which can then be compared to the PDC-corrected light curves.

1.4 Scientific Goals

This specific autoencoder technique has not been applied to TESS light curves for the explicit purpose of removing noise, so the results of this project can shed light on whether this technique is effective at denoising light curves. Updating TESS’s systematic noise corrections pipeline has the potential to reveal fainter transits, especially smaller planets around noisier stars that inherently have a lower signal. By improving the systematic error floor, we can detect more planets and improve the measurement precision of existing planets.

CHAPTER 1. INTRODUCTION

This project has the potential to provide the community with a novel pipeline that could produce cleaner TESS light curves and find more exoplanets. If the method improves noise reduction by even 1 or 2 percent, it nonetheless unlocks an entirely new regime spanning thousands of exoplanets that was previously inaccessible, including smaller exoplanets around noisier stars that are less studied because of their fainter signals. As we build and refine this pipeline to improve noise corrections, we will be able to access new targets in this noisier regime, including habitable-zone planets and Earth-like worlds. However, many of these planets orbit young, active, low-mass stars which yield noisy light curves with low SNR. We hope to probe this regime of data better and search for more habitable-zone worlds around noisier stars. This project would also improve our understanding of how machine learning can be utilized in astronomy as a tool for improving data quality, which is applicable to other large astronomical surveys.

1.5 Outline

Chapter 2 discusses the method of building and testing the pipeline. We describe how we select and preprocess TESS light curves, construct the autoencoder, and train our algorithm on the TESS data. Chapter 3 presents the cleaned light curves from our most current training iteration, comparing their properties to the PDC-corrected light curves. We focus on a dozen light curves with known planets in order to measure how well our algorithm preserves those signals, as planetary signals are the ones we hope to reveal. Chapter 4 discusses ways to improve the pipeline and future steps to take.

Chapter 2

Methods

2.1 Preprocessing Pipeline

2.1.1 Sector Selection

Because systematic noise occurs across each sector, our proposed pipeline must create unique autoencoders to perform noise subtraction on each sector. To start, we select an older Sector, Sector 3, to test the pipeline. Eventually, we hope to compare our pipeline’s effectiveness between Sector 3 and a newer sector with some of the same targets. We use the 2-minute cadence data so we can directly compare our light curves to the PDC 2-minute cadence data.

The light curves require several steps of preprocessing to be prepared for training. Figure [2.3](#) depicts 10 light curves as they are passed through the full preprocessing pipeline.

2.1.2 TESS Quality Flags

TESS flags certain data points with Data Anomaly flags based on various different kinds of errors that can occur with the telescope. Some points are flagged because of issues with the telescope itself, while others are assigned based on the SPOC pipeline results. TESS encodes these bad quality flags as bits, with 12 bits in total that represent different reasons that the datapoint was flagged. For example, if Bit 1, Bit 4, and Bit 7 were activated, the quality flag will be 000001001001, or 49.

Bits 1, 2, 3, 4, and 6 were assigned based on spacecraft telemetry, meaning that those points were bad quality due to errors with the instrument. We therefore throw out these points by setting them to the median and keep track of their location so that we can ignore them.

We also aim to remove Bit 12, which is flagged because of stray light across the telescope. Unlike the previous bits, this flag is assigned to different points across different light curves. We set any point flagged with Bit 12 to the median value of the light curve, keeping track of their location in individual light curves so we can remove them after training.

However, we retain data points flagged as bad quality by the SPOC pipeline, since our goal is to test our own noise subtraction method on raw light curves without prior SPOC-based corrections. With the autoencoder, we hope to perform a more flexible and accurate noise subtraction, which might be able to preserve points that SPOC threw out.

2.1.3 NaN Preprocessing

The PDC light curves are flagged with thousands of NaN values. When we initially trained light curves, we set the NaN values that existed in the light curves already to the median, so that we could train over light curves of the same length. However, approximately 3000 data points at the beginning and end of the PDC light curves were sliced off and set to NaNs, likely because of bad quality data.

Visual inspection reveals large outliers at the start of data collection. To maintain consistency with the PDC pipeline and avoid training biases from outliers, we remove these initial and final NaN values. This slicing also made it easier for the autoencoder to learn light curve features without having to deal with large outliers in the data.

Another large slice of NaN values occurs in the middle of the dataset for all PDC light curves, when TESS transfers its data to Earth. If 10% or more of these consecutive points in the middle are set to NaN, then we flag it as a point to remove in our light curves. However, since this slice of NaNs occurs in the middle of our data, we leave these bad points in for training, and remove this block at the very end.

There are also many NaN values that appear in the raw data. However, not all of these NaN values appear in every single light curve, and it is crucial to maintain the same number of inputs for the autoencoder to train. We therefore keep track of every NaN value in our light curves and set them to the median so that we can set them back to NaNs at the end of training.

2.1.4 Sigma-Clipping and Normalizing

Because there are large outliers still present in the data after preprocessing, we consider sigma-clipping the data, removing points that were more than 3 sigma above the median. Outliers can negatively impact the autoencoder training because they skew the loss function – when computing MSE, a large outlier will create a spike that will make it difficult to update weights across all light curves. However, removing outliers below the median could potentially destroy transit signals.

In the end, we decide not to sigma-clip our data before training to avoid potentially removing real transits or astrophysical signals. However, we revisit the issue of sigma-clipping in Section 3.3.2.

We also normalize the light curves, which is necessary to scale them for training. Large differences in the scale of the light curves would make it difficult to minimize the loss across all light curves. We normalize our light curves to have a median of 0 and a standard deviation of 1 according to this function:

$$X_N = \frac{X_i/\bar{X} - 1}{\sigma/\bar{X}} \quad (2.1)$$

where X_i are the light curve fluxes, $\bar{X}$ is the median value, and σ is the standard deviation. First, we scale all the data points by the median, effectively centering them around 1. We subtract 1 to center the data points at zero. Finally, we divide by the standard deviation of our new data set, which is also scaled to the median so that it reflects the new standard deviation of our light curves. Dividing by this standard deviation scales our light curves to have a standard deviation of 1.

2.2 Constructing the Autoencoder

We use the machine learning library `pytorch` to help build our autoencoder and perform training. We design the autoencoder to have either two or three layers in total. When we construct the autoencoder, we introduce nonlinearity in the hidden layers, but maintain linearity for the layer that creates the latent space or the reconstruction. The nonlinear layers will be scaled by the activation function, so we reintroduce linearity in the final layer of the encoder and the decoder so we can reconstruct and represent our inputs properly. The first and final layers of the autoencoder are linear so that the data is more easily interpretable and representative of the inputs. Figure 1.7 demonstrates this structure.

We select the 50% of light curves with the lowest intrinsic scatter for training and testing. This choice ensures that the autoencoder learns systematic noise patterns rather than try to fit white noise. By minimizing the effect of intrinsic stellar or planetary signals, we optimize the model’s ability to extract systematic noise while preserving real astrophysical features.

The `pytorch` library enables efficient training by allowing light curves to be loaded in batches, reducing memory usage and stabilizing gradient updates. Mini-batch training helps avoid noisy updates to weights and improves convergence speed, especially when working with large datasets.

2.3 Training

To train an autoencoder, we divide the dataset into a training set and a test set, split 80%/20%. The autoencoder pipeline is applied to both sets, and the loss function is computed on each. However, gradient descent and backpropagation to update the model weights are only performed using the training set. The test set serves as a gauge for how well the model generalizes to unseen data. For example, if the training loss decreases but the test loss does not, the model is likely overfitting to the training set.

We chose the 50% quieter light curves for our training and test set, since the data with less intrinsic noise will be easier to extract the systematic noise from and minimize loss over.

The `pytorch` library enables efficient training by allowing light curves to be loaded in batches, reducing memory usage and stabilizing gradient updates. Training in smaller batches helps avoid noisy updates to weights and improves convergence speed, especially when working with large datasets.

2.3.1 Binning

Before training, we bin our data by averaging every 5 consecutive points. While not strictly necessary, binning the data increased the speed and efficiency of training. Since the input size is based on the number of points in each light curve, starting with an input size of binned light curves increased the computational speed of the autoencoder.

Binning also decreases variance by reducing noise and smoothing the light curves, making it easier for the autoencoder to learn light curve features better. Binning also

averages over time-series data, which could potentially damp or obscure shorter transits, enabling the autoencoder to preserve those signals even better. However, binning 2-minute cadences by 5 would not ignore most transits.

2.3.2 Random Search

Once the autoencoder and dataset are initialized, we optimize its hyperparameters using random search. Hyperparameters are tunable parameters that control the architecture and training process of the neural network, such as learning rate, number of layers, number of neurons per layer, batch size, activation function (which we set to be [1.4](#)), and optimizing function (which we set to be gradient descent with momentum).

We use a default momentum value of 0.9 and a batch size of 128 to train. The hyperparameters which affect autoencoder performance most are the size of the neural network and the learning rate. We start by using a random search to optimize for a learning rate between 1 and 10^{-5} , with layer sizes of 32, 64, 128, 256, 512, 1024, and 2048. We train with only two layers, although training with more layers is possible. We use random search for our initial optimization.

Random search is chosen over grid search because it efficiently explores a wide range of hyperparameter values without requiring an exhaustive search of all possible combinations. Random search often finds near-optimal solutions with fewer iterations, making it computationally efficient. While Bayesian optimization could offer a more targeted search, random search provides a straightforward and effective approach given our high-dimensional hyperparameter space.

The number of epochs sets how many steps we iterate over when we train an

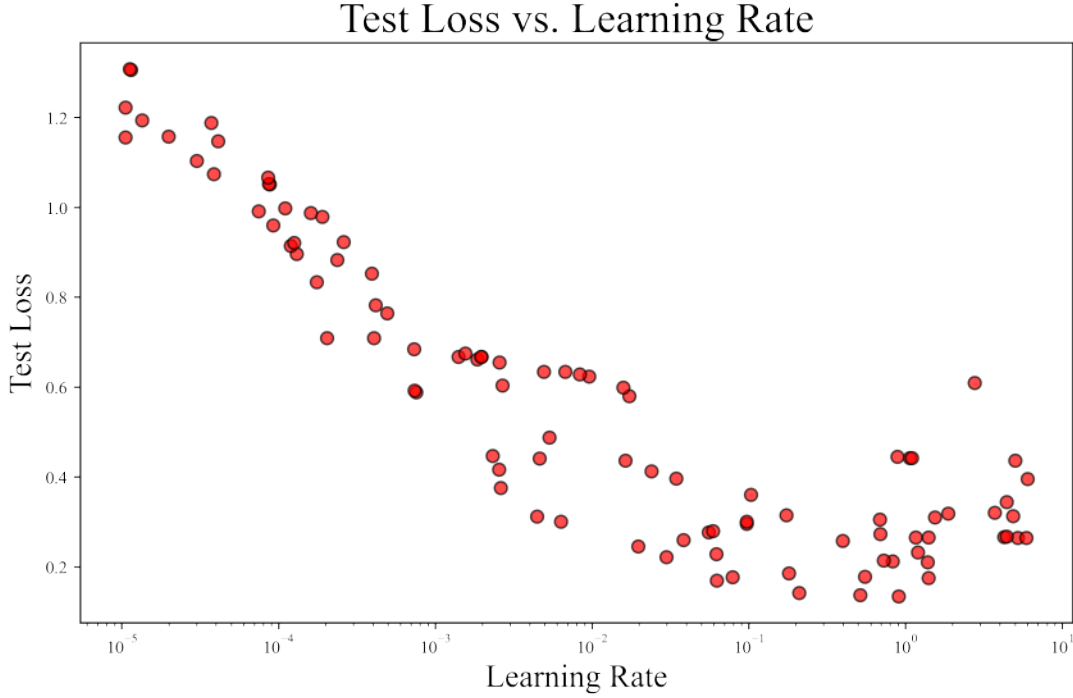


Figure 2.1: We optimize for the best learning rate by recording the final test loss after each run and selecting the learning rate that leads to the lowest loss.

autoencoder. During random search and grid search, we train with around 50 epochs, which is generally sufficient to see how well the autoencoder performs. Once we find the parameters that minimize loss, we train over 200 epochs to allow the autoencoder to iterate more and produce more accurate reconstructions.

We plot the final test loss as a function of different learning rates in Figure 2.2. Interestingly, we find that learning rate’s effectiveness was relatively independent of layer size. The autoencoder’s test loss was minimized most effectively with learning rates between 0.05 and 1. We then perform a smaller grid search to fine-tune the learning rate, selecting first and second layer sizes of 512 and 256, as the models with the lowest test loss had these layer sizes.

Parameter	Value
Input Size	2920
Number of Layers	2
First Layer Size	512
Second Layer Size	256
Batch Size	128
Learning Rate	0.4722
Momentum	0.9
Epochs	200
Final Train Loss	0.1158
Final Test Loss	0.1377

Table 2.1: Final autoencoder parameters that minimized test loss.

Table 2.1 shows the final autoencoder parameters that minimized loss.

2.4 Reconstruction and Subtraction

After training our autoencoder and selecting the best hyperparameters through random search, we pass our entire sector of light curves through the autoencoder to reconstruct their systematic features. Since we bin the light curves by a factor of 5 before training, the reconstructions have the same reduced length as the binned inputs. To restore the original cadence, we use a cubic spline interpolation from the `scipy` library, fitting a cubic polynomial to interpolate between all points to ensure that the reconstructed light curves match the full length of the original dataset.

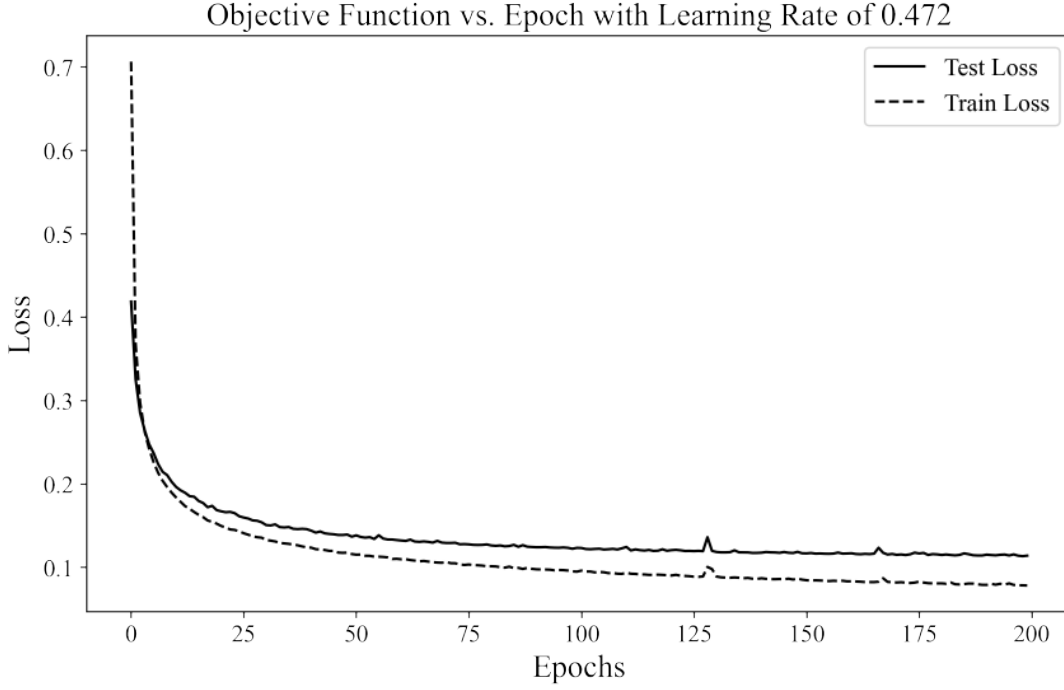


Figure 2.2: When we train our autoencoder, we record the loss over each step or epoch, with the aim of minimizing loss. We expect the training loss to be slightly lower than test loss since the model is iteratively updating its weights based on the training dataset.

Finally, we subtract the reconstructed light curves – which should contain only features of systematic noise – from the original light curves.

2.4.1 Postprocessing

To compare our light curves to PDC, we unnormalize our light curves by reversing the normalization function:

$$X_i = \left(X_N \cdot \frac{\sigma}{\bar{X}} + 1 \right) \bar{X} \quad (2.2)$$

CHAPTER 2. METHODS

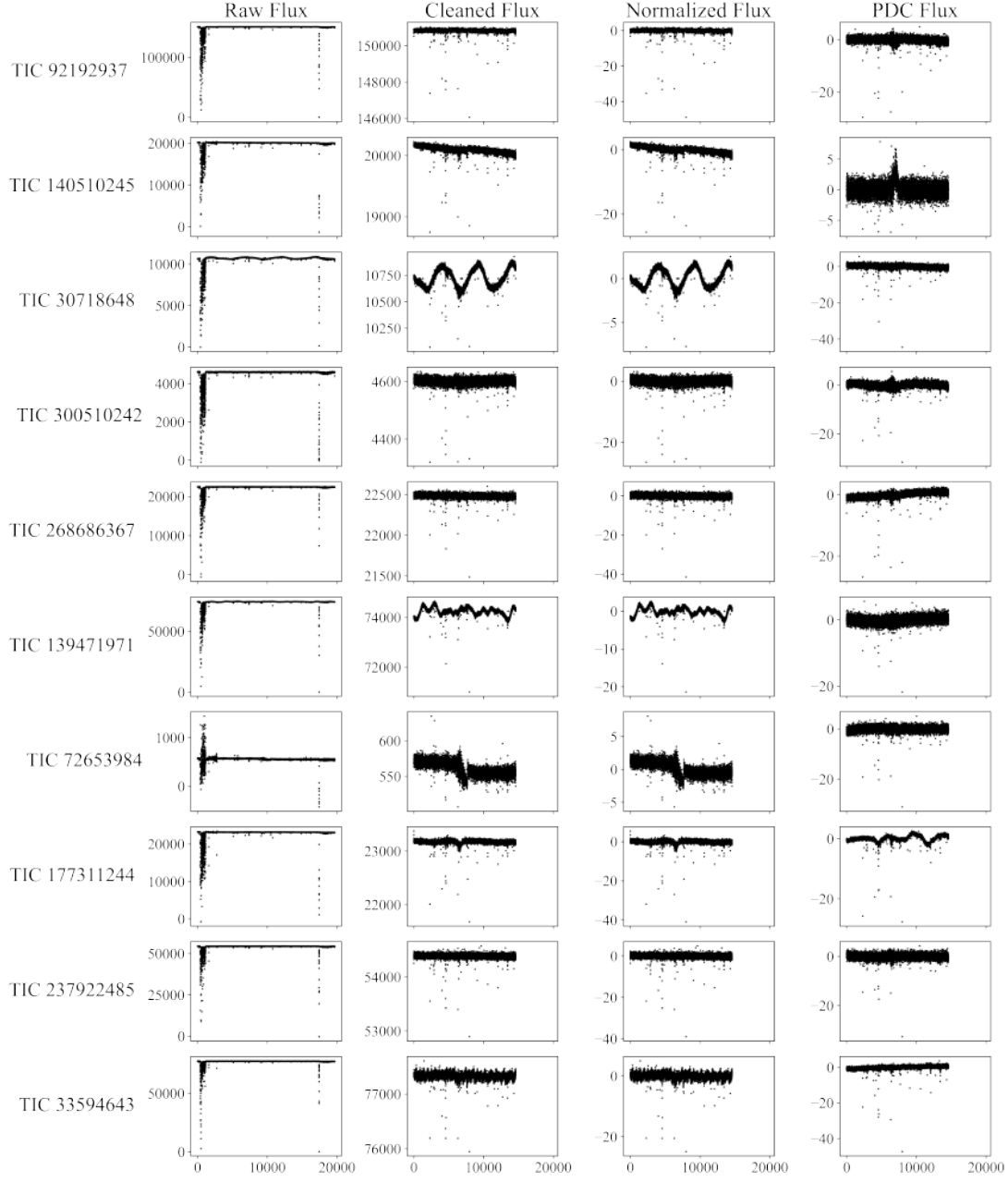


Figure 2.3: Preprocessing pipeline applied to TESS light curves before training, with the PDC-corrected light curves in the right column for comparison. The cleaned flux has bad quality points removed, while the normalized flux has adjusted all the light curves to have a mean of 0 and a standard deviation of 1.

CHAPTER 2. METHODS

where X_N is the normalized light curve, σ is the standard deviation, and $\bar{X}$ is the mean of the original light curve.

We also reintroduce NaN values where we had replaced them with the median. This includes insert NaNs at the beginning and end of the light curves where we clipped the data due to quality concerns, in the middle of the light curves where the spacecraft passes behind the Moon, and at all points flagged with quality bits corresponding to bad data.

Chapter 3

Results

3.1 Subtractions

After training our autoencoder and optimizing for the best parameters, we can pass our light curves through the autoencoder to get our reconstructions. Figure [3.1](#) demonstrates how our binned reconstructions compare to the original light curves in a randomly generated sample. We find that our reconstructions match our original light curves nearly perfectly, indicating that they may be reconstructing more than just noise but also astrophysical signals. While the autoencoder appears to effectively learn the features of the light curves, learning the features too well is also a concern, as that would lead to real signals in the data being removed.

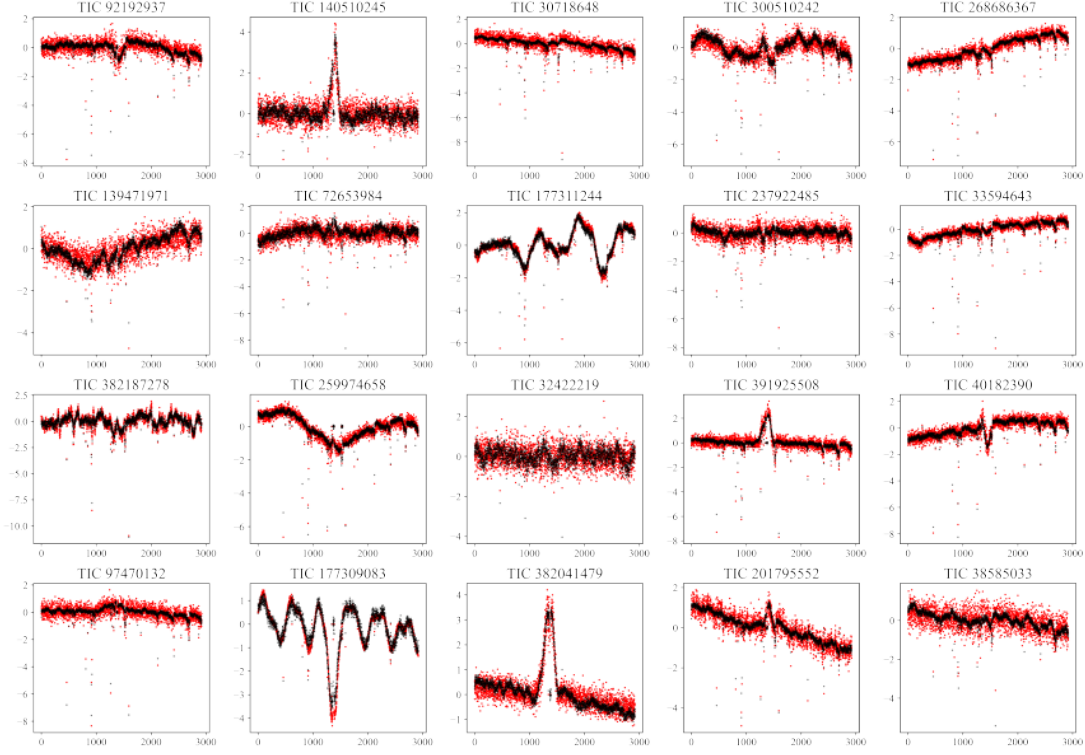


Figure 3.1: We can visually compare our reconstructed light curves to their inputs. The light curves shown here are a random sample from the entire dataset, which includes the training and test sets. The red color indicates the original, raw light curve, while the black curve indicates the reconstructions.

3.2 Comparing RMS

We also compare the RMS of our light curves with the PDC-corrected light curves in Table 3.1. We find that our pipeline produces light curves with 0.3% lower RMS. Figure 3.2 shows a distribution of the RMS for our pipeline and PDC – the RMS distribution is nearly the same.

Reducing the RMS of the light curves is a major goal of the project, since it would allow us to access more planets at lower SNR. However, reducing the RMS cannot come

CHAPTER 3. RESULTS

Method	RMS Mean	RMS Standard Deviation
Autoencoder	98510.36	606983.72
PDC	98806.37	604569.81

Table 3.1: Comparing RMS between the autoencoder and PDC yields a 0.3 % improvement in the autoencoder noise correction pipeline.

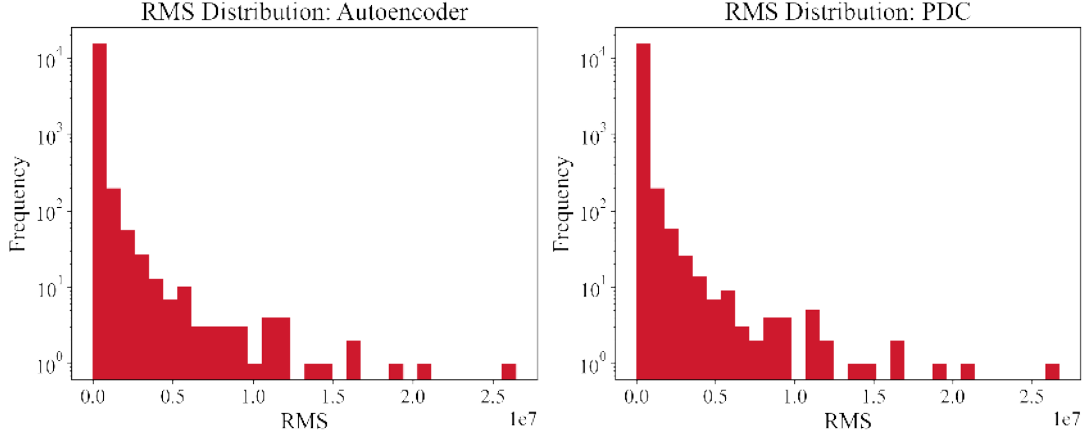


Figure 3.2: The RMS of our light curves compared to the RMS of the PDC-corrected light curves.

at the expense of removing actual transit signals. Subsequent sections examining our pipeline’s version of light curves with planetary transits indicate that this signal removal is in fact a large problem with our current model.

CHAPTER 3. RESULTS

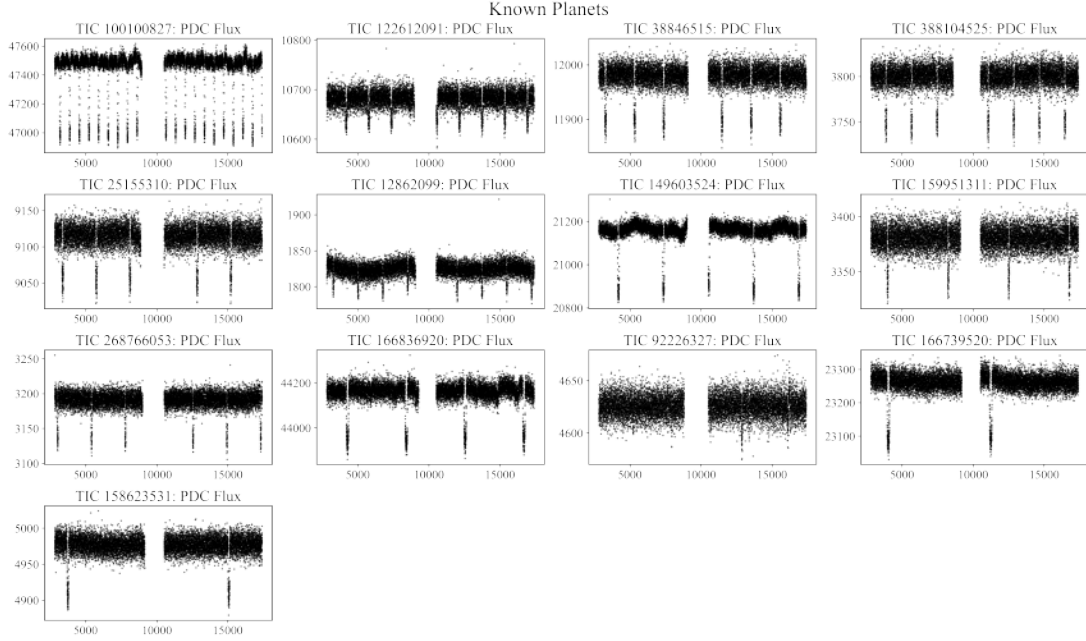


Figure 3.3: PDC-corrected light curves of known planets prior to TESS. The transit signals are clearly visible.

3.3 Comparing TOIs

3.3.1 Known Planets

We first compare all of the known planets in Sector 3 across PDC and our autoencoder pipeline. Known planets are planets that were discovered and confirmed prior to TESS.

3.3.2 Sigma-Clipping

Comparing the PDC-corrected light curves (Figure 3.3) with the autoencoder-corrected light curves (Figure 3.4), it becomes clear that the autoencoder fails to preserve the transit signals well. However, there were several large outliers present in the

CHAPTER 3. RESULTS

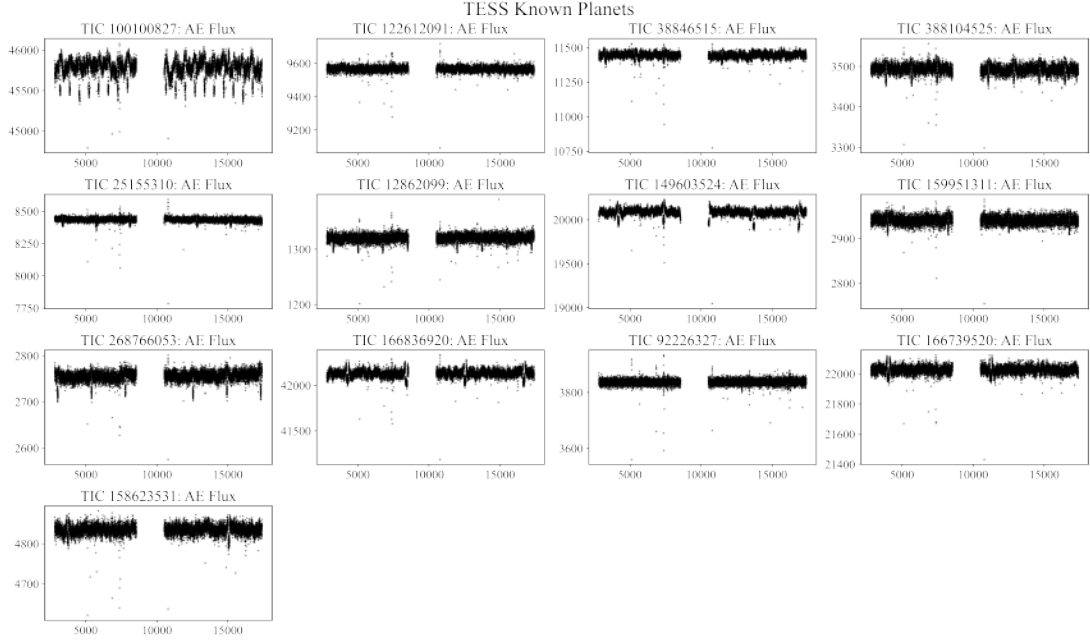


Figure 3.4: Autoencoder-corrected light curves of known planets prior to TESS. The transit signals cannot be seen, although this may be due to large outliers.

autoencoder-corrected light curves that may have been skewing the visual. In the original preprocessing pipeline, we chose not to sigma-clip in order to avoid removing transits. However, the initial results of the autoencoder suggest that sigma-clipping may be necessary.

Therefore, we sigma-clip our data to remove points more than 7 sigma away from the baseline – we choose such a high value to sigma-clip after inspecting the light curves after sigma-clipping and observing whether a large amount of points were excessively clipped. However, transits may be many sigma away from the median, so a more generalizable method for sigma-clipping will be necessary for the final pipeline.

Inspecting the sigma-clipped data reveals that the autoencoder does preserve the transit signals to some extent. However, the baseline flux and the depths of the

CHAPTER 3. RESULTS

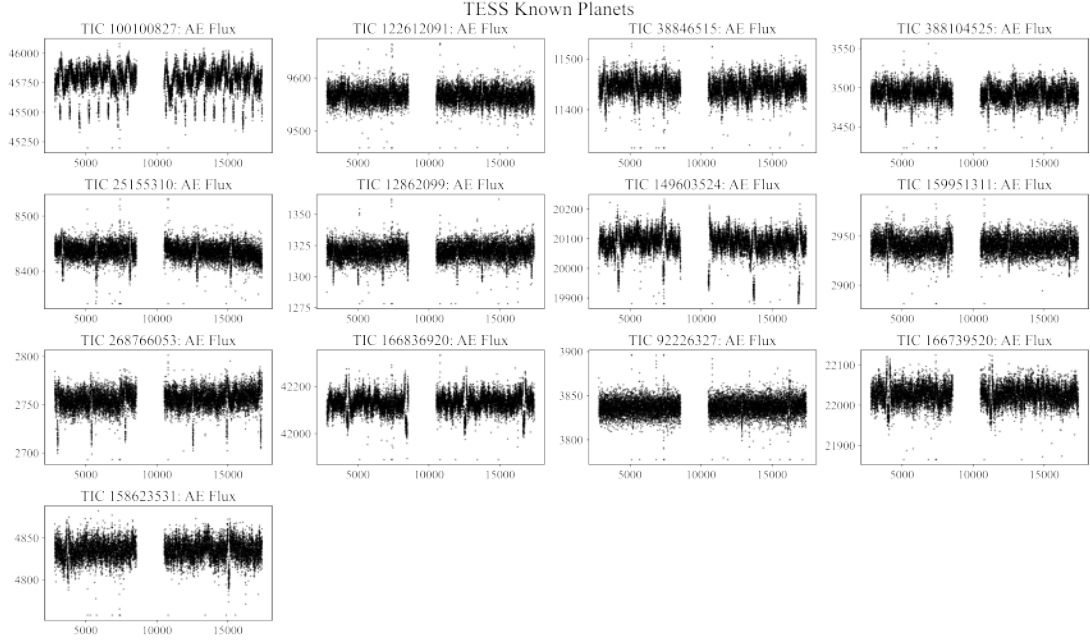


Figure 3.5: We sigma-clip our autoencoder-corrected light curves to better assess how well it preserves transit signals.

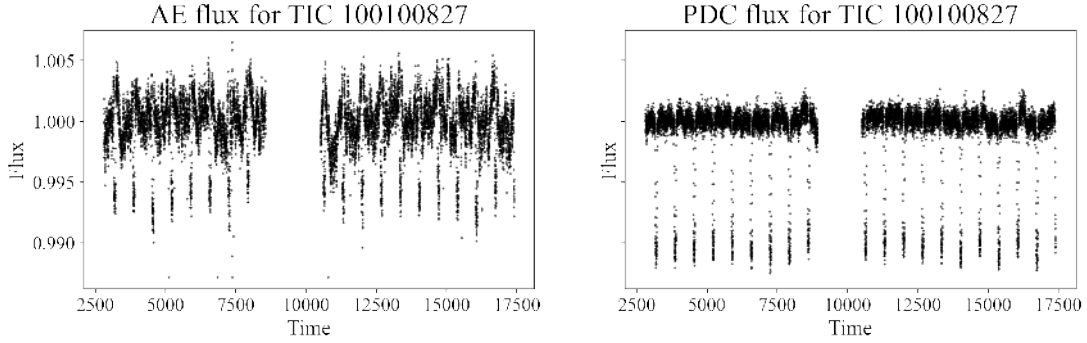


Figure 3.6: WASP-18 b was discovered in [Hellier et al. \(2009\)](#). The planet is a large gas giant with a mass of 10.2 days and a short orbital period of 0.9 days.

autoencoder-corrected light curves appear different. The most crucial aspect to preserve is the depth and shape of the transit signals, so we inspect two planets in our sample of known planets and normalize them to compare their fractional depths.

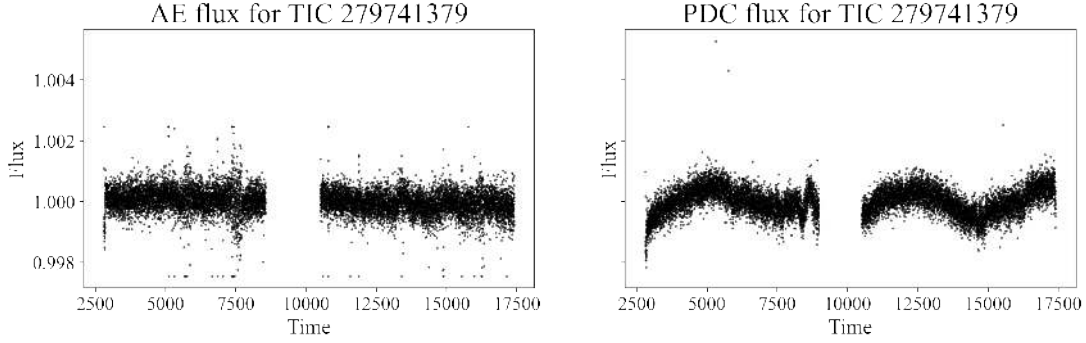


Figure 3.7: WASP-139 b was discovered in [Hellier et al. \(2017\)](#). The Super-Neptune orbits its star every 5.9 days.

We specifically compare the transits of WASP-18 b (Figure 3.6) and WASP-139 b (Figure 3.7), a short-period hot Jupiter and super-Neptune, respectively. We normalize the light curves the given period to compare their fractional depths. Both autoencoder-corrected versions of the transits do not preserve the transit depth but make it shallower, indicating that the autoencoder is at least partially reconstructing and therefore modifying the transit signal. The autoencoder also removed some intrinsic variability in WASP-139 b’s light curve, indicating that it is able to capture and subtract potentially stellar signals as well.

We also phase the light curves (Figure 3.8 and 3.9) to see how well the autoencoder is preserving the transit over time. The autoencoder appears to preserve the duration and timing of the transits well; however, the baseline of the phased transit is clearly different than the baseline of the PDC-corrected light curves, biasing the transit depth.

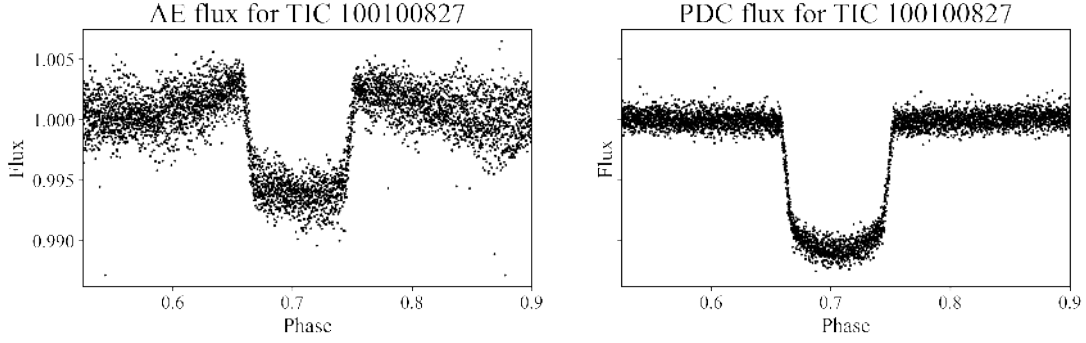


Figure 3.8: The phase diagram of WASP-18 b demonstrates that the fractional transit depth is not preserved well, although the transit timing is maintained.

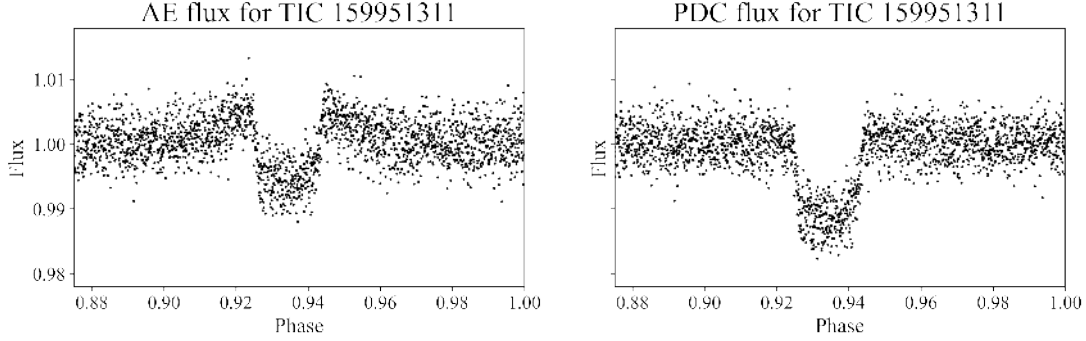


Figure 3.9: The phase diagram of WASP-139 b also indicates that the transit depth is not well-preserved by the Autoencoder.

3.3.3 Confirmed Planets

We also look at transits of planets confirmed by TESS to compare the PDC-corrected light curves (Figure 3.10) to our light curves (Figure 3.11), sigma-clipping these light curves as well. The autoencoder-corrected version of the light curves also have shallower transit depths or hardly any identifiable transits. The autoencoder also appears to subtract all signs of intrinsic variability in the light curves that PDC maintains.

We compare the transits of HD 18599 b (Figure 3.12) with the PDC-corrected flux.

CHAPTER 3. RESULTS

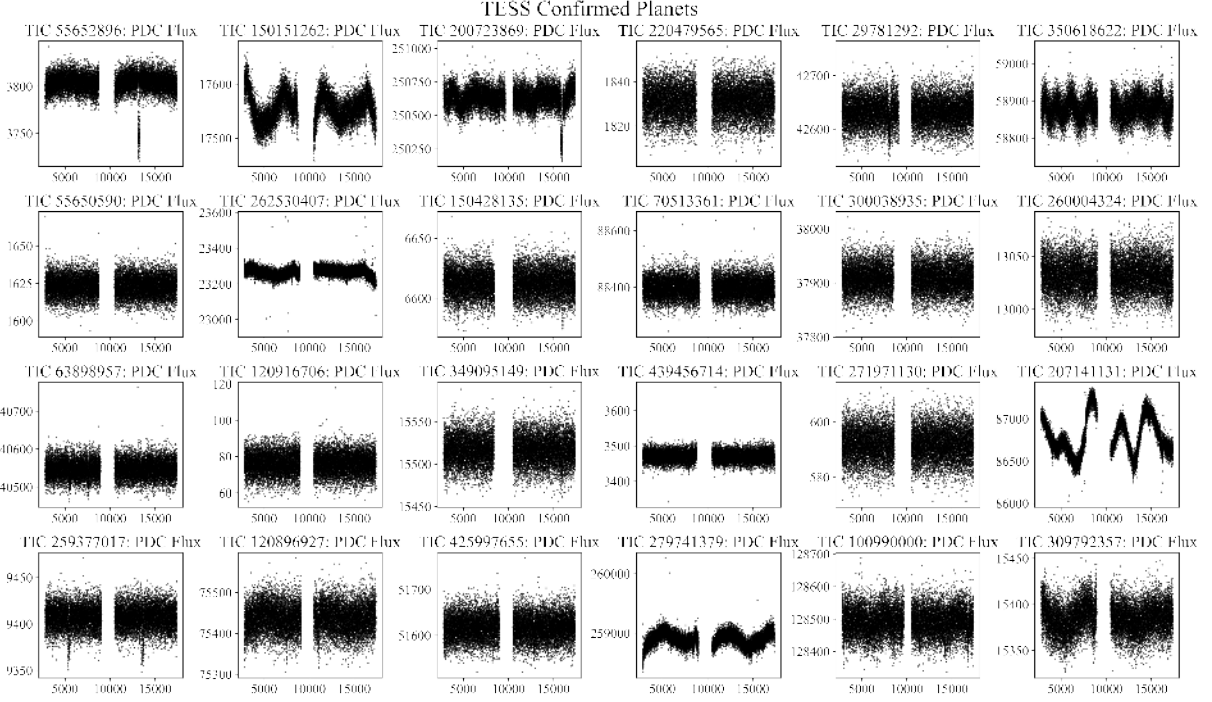


Figure 3.10: PDC-corrected light curves of planets confirmed with TESS.

The PDC-corrected light curve contains stellar activity that [Desidera et al. \(2023\)](#) models in their discovery paper. Notably, the autoencoder-corrected flux does not preserve this variability, but subtracts it completely. This becomes clear in the phase diagram (Figure 3.13), where the transit signal in the autoencoder-corrected light curve is much more clear due to the absence of stellar variability.

CHAPTER 3. RESULTS

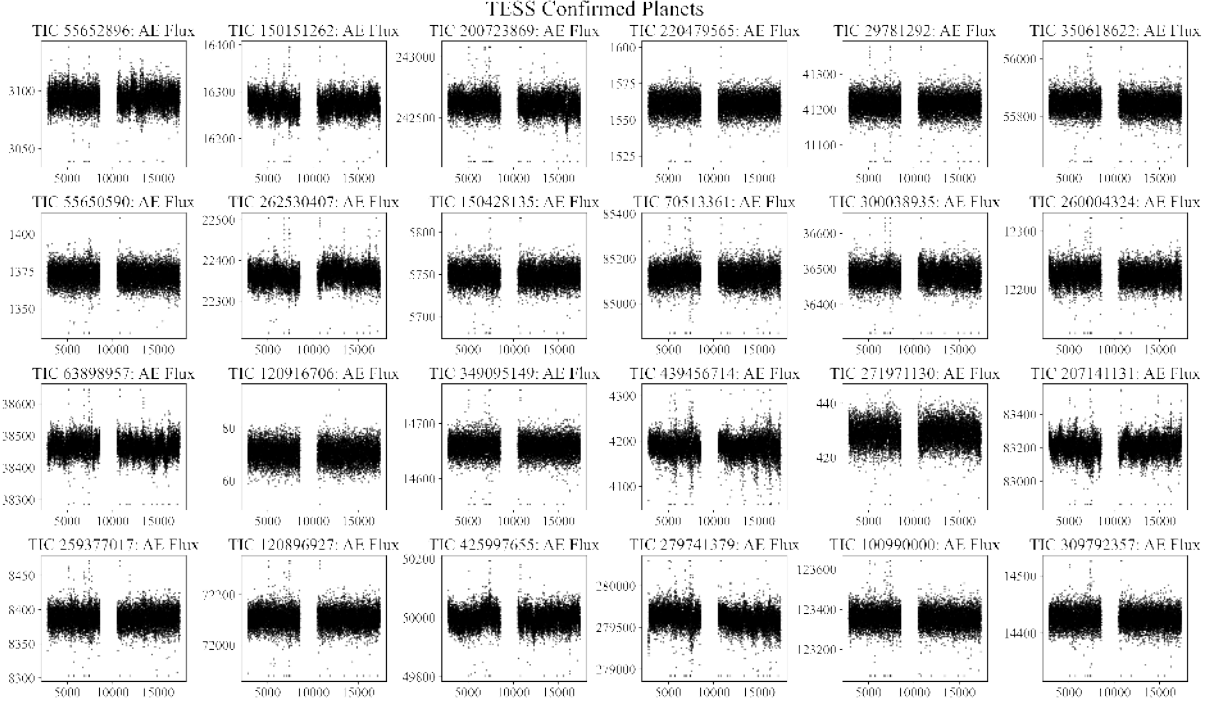


Figure 3.11: Sigma-clipped light curves of planets confirmed with TESS that were noise corrected using the autoencoder. The transit depths are not well-preserved.

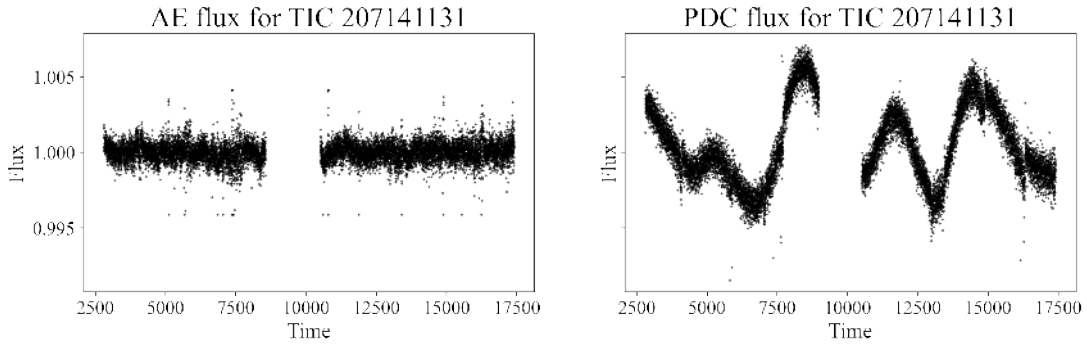


Figure 3.12: HD 18599 b, discovered by [Desidera et al. \(2023\)](#), is a young, Neptune-like planet that orbits an active K-type star every 4.1 days. The star’s activity can be seen in the transit.

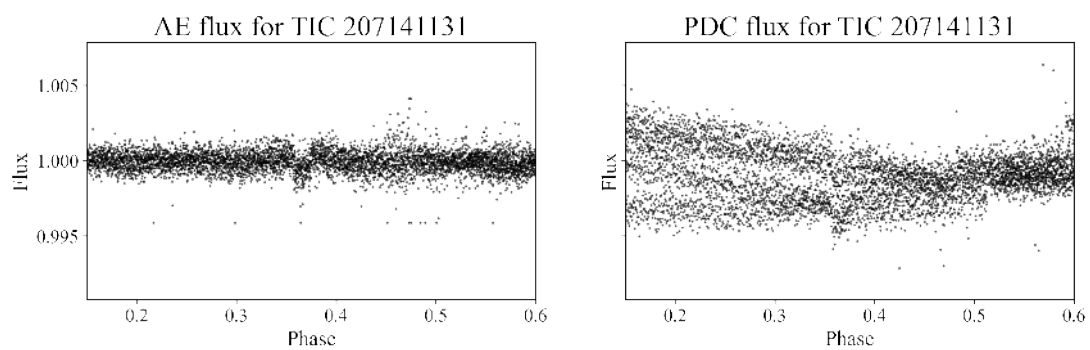


Figure 3.13: Phase diagram of HD 18599 b. The autoencoder-corrected flux has a clearer transit in the absence of stellar activity that it was able to remove. However, it remains unclear whether the depth of the transit was well-preserved.

Chapter 4

Discussion

4.1 Improving Our Pipeline

4.1.1 Preserving Transit Signals

While the autoencoder effectively captures and reconstructs features of the light curves, it appears to have worked too well – partially capturing and modifying transit signals in the process. This unintended behavior raises concerns about the extent to which the autoencoder is able to differentiate between stellar variability, instrumental noise, and astrophysical transits. The flexibility of the autoencoder seems to allow it, in this first iteration, to capture to some degree the unique planetary signals across light curves.

We can explore various different preprocessing techniques to help the autoencoder pipeline ignore and ultimately preserve transit signals. Masking the transits somehow before training might prove effective, although doing so might also defeat the purpose

CHAPTER 4. DISCUSSION

of using the autoencoder to find new transits if we have to know where the transits are to mask them. We could also explore sigma-clipping or outlier rejection – removing large outliers or potential planetary signals in the data prior to training so that the autoencoder can learn only the noise. We can also consider binning the data further or changing the time scale of the data somehow to ignore or pass over transit signals.

In addition to preprocessing, tuning the autoencoder’s parameters may improve its ability to distinguish between transits and noise. For example, constraining the model’s capacity – such as by increasing the bottleneck across layers – could reduce its ability to reconstruct finer details, forcing it to focus on large-scale systematics rather than individual astrophysical signals. Our model’s latent space contained 256 neurons, while PDC uses only 8 CBV vectors. Training the model on a much smaller latent space, such as 10 neurons, could effectively enforce a bottleneck that allows the autoencoder to learn only the systematics.

Because the autoencoder seemed to partially capture the transits, one potential limitation is that it may be impossible to configure it to ignore large or deep transits. However, focusing ignoring and preserving shallower transits could be an effective use of the autoencoder, if preserving deeper transits proves to be impossible. To focus on shallower transits, we could ensure that all the shallow transits in Sector 3 are within the training or test set, while the deeper transits are excluded. We could also consider implementing a Median Squared Error loss function instead of Mean Squared Error, so that the computed loss is less affected by outliers such as transits – thus not penalizing the model for not learning transit features.

Figure 3.13 demonstrates the uselessness of the autoencoder as a tool for removing

stellar variability. However, there are other ways of removing stellar variability from transit signals. The pipeline’s effectiveness can be better quantified by its ability to remove these signals while also preserving transit properties.

4.2 Future Steps

4.2.1 Preprocessing

While producing a pipeline that can ignore transit signals is the most pressing concern to address, there are other areas for improvement that we can explore in order to build a better pipeline. The end goal of this thesis is to see whether an autoencoder can outperform the current SPOC pipeline in noise subtraction while preserving a greater portion of the dataset. If successful, this approach could increase the amount of usable data in the TESS archive.

One way to demonstrate this improvement is to perform less preprocessing and get similar results, effectively preserving more of the dataset than SPOC was able to preserve. We can assess the autoencoder’s effectiveness by retaining more data points at the beginning and end of each light curve, which are typically removed due to large outliers. These points were excluded both because they introduced significant variance during training and because the PDC pipeline removed them in its preprocessing. However, if the autoencoder is capable of subtracting noise from these regions effectively, it could recover several additional days of observations per sector.

Another way that our autoencoder could improve the breadth of the TESS dataset

CHAPTER 4. DISCUSSION

would be by training on light curves that have not had quality flags removed. While SPOC discards these points outright, an autoencoder-based approach may be able to identify and correct systematic noise within them, allowing more of the dataset to be retained for analysis.

We can also reconsider the handling of NaNs. Currently, we set all NaNs to the median, then remove them at the end. However, we can instead mask the NaNs in the autoencoder’s loss function instead, adding a weight feature to MSE where the weight is 0 if the point is a bad pixel or NaN, and 1 if it is not. This change would allow the model to handle NaNs without requiring additional steps that could potentially modify the data inaccurately.

4.2.2 Training

We bin our training and test set to reduce variance and increase the speed of training. However, we could train on a non-binned dataset as well, which may enable the autoencoder to extract systematic features that were smoothed over with the binning.

We may also want to explore hyperparameter space further. We only train an autoencoder with two or three layers. However, deeper networks could enable more flexible learning and optimize loss better. We can also tune our momentum value or apply different optimizers or non-linear activation functions, although the improvement may only be marginal.

When we selected the training and test set, we chose the 50% of stars with the least outliers, since it might be easier for the autoencoder to pick up on systematics in light curves with less intrinsic scatter. A better metric for selecting the quietest light curves

CHAPTER 4. DISCUSSION

would be to compute a rolling RMS – taking the RMS over 1-hour time intervals and averaging those values to compute a binned RMS – and select the 50% of light curves with the lowest rolling RMS.

Rolling RMS is a better measure of the light curve’s intrinsic noise since it is less sensitive to long-term variations in stellar variability. While stellar variability can impact SNR, planetary signals can still be extracted over long-term stellar variability. However, noise on a shorter 1-hour timescales, which is quantified by the rolling RMS, decreases SNR and makes it harder to detect transits. Rolling RMS is therefore less affected by stellar variability and will quantify which light curves have better SNR, or better detectability for both planets and systematics.

4.2.3 Comparing Pipelines

We compare our autoencoder’s performance to PDC by analyzing the RMS of the corrected light curves. In addition to the overall RMS, we can also compute the rolling RMS to assess how well each pipeline reduces noise on different timescales. Comparing the rolling RMS of the autoencoder-corrected light curves to PDC-corrected light curves is a more effective metric for assessing how well the autoencoder decreases SNR by removing systematic noise while preserving astrophysical signals. Figure 4.1 shows the rolling RMS, also called a 1-hour CDP, as a function of magnitude, for a random sector of TESS. Intrinsically brighter stars will have lower RMS, since the SNR will be higher and the scatter due to noise will impact data quality less than it would a fainter star. Plotting RMS over magnitude for the PDC-corrected and autoencoder light curves is an effective way to compare how our pipeline improves noise correction.

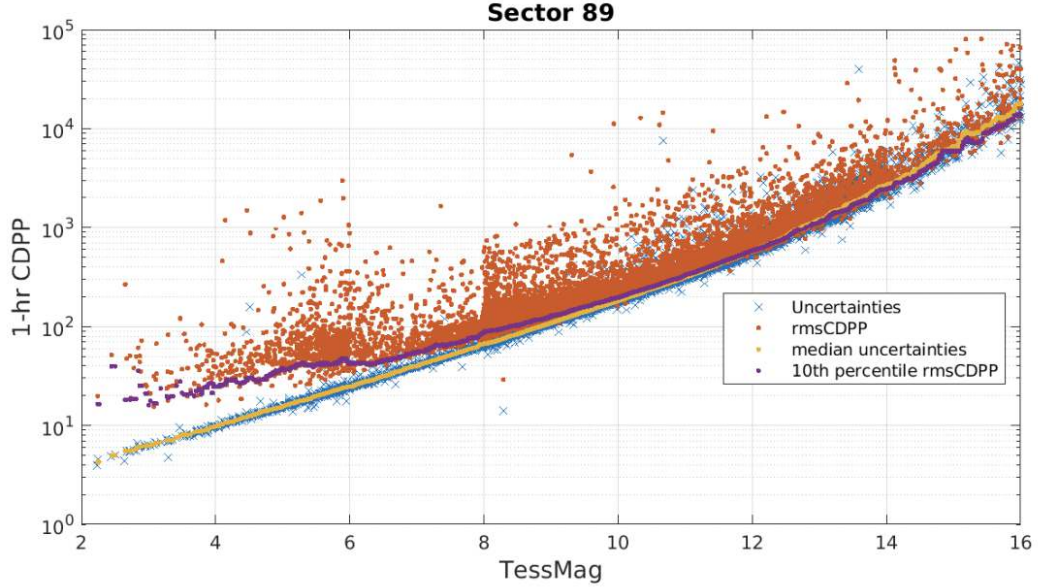


Figure 4.1: Rolling RMS as a function of magnitude is pictured here for Sector 89. The rolling RMS is a better measure of noise, since it is less sensitive to long-term stellar variability.

We can also test the autoencoder’s ability to preserve transit signals by injecting synthetic transits into the light curves before applying the noise correction pipeline. By comparing the recovered transits to the injected signals, we can quantify how well our pipeline maintains transit depth, duration, and shape. This approach provides a direct measure of the autoencoder’s performance in preserving planetary signals while removing noise.

This injection recovery technique is a particularly powerful way to ensure that our autoencoder is not modifying transits. If we can determine a way to allow it to ignore transit signals without directly masking transits that we know exist, then injection recovery will be the primary way that we can test its effectiveness and ensure that we can trust the planetary parameters we do extract.

4.2.4 Searching for Transits

Once we are satisfied with our autoencoder pipeline’s ability to subtract noise and preserve transits, we can run a Box Least Squares (BLS) periodogram to search for transit signals across the entire dataset. If the autoencoder improves noise subtraction by 1% or 2%, we should expect to find hundreds, if not thousands, more TOIs in the TESS data.

4.2.5 Removing Noise From Subsequent Sectors

If the pipeline works effectively for Sector 3, then we hope to train it on subsequent sectors. Each sector will require a unique autoencoder to correct for its own noise features. In particular, we may want to focus on a newer sector next to compare the effectiveness of our technique to Sector 3. We may also select a sector that has the same TOIs, so that we can directly compare our pipelines against the same source in two different sectors.

4.3 Conclusions

Our noise correction pipeline remains a work in progress. While the autoencoder demonstrates promise through its ability to effectively reconstruct light curve features and stellar activity, the pipeline is fundamentally unusable if it cannot preserve transit signals. The goal of building this pipeline is to reveal and amplify planetary signals by removing noise – modifying or damping these signals would be catastrophic, as astronomers glean the fundamental properties of planets from precisely measuring

CHAPTER 4. DISCUSSION

features of the transit light curves.

The autoencoder appears to have reconstructed the light curves too well. It blindly learned the crucial features of the light curves, including, at least partially, their planetary signals. If the model inadvertently suppresses planetary transits, it could erase the very signals we aim to detect. The autoencoder’s ability to learn these transits raises an important question: How can we be sure that our autoencoder is not modifying our signals? Can we trust it?

The autoencoder’s performance also emphasizes the blindness of certain machine learning algorithms – it cannot physically distinguish between stellar signals, planets, and systematic noise features, and it cannot be a substitute for human vetting. However, it can serve as a highly useful and advanced tool, if we utilize it right.

Moving forward, we hope to rigorously constrain our autoencoder noise correction pipeline to remove enough systematic features to improve SNR while preserving transits, ensuring that it remains sensitive to astrophysical signals while filtering only unwanted systematics. These improvements will require careful vetting through extensive training, synthetic transit injections, and comparisons against established pipelines.

The autoencoder must strike a delicate balance: precise enough to remove noise, yet constrained enough to preserve planetary signals. The success of our pipeline depends on refining the model architecture, training pipeline, and preprocessing techniques to both improve performance and prevent overfitting. Ultimately, if we can develop a pipeline that surpasses existing noise removal methods while revealing transit signals, we will offer the community a powerful tool capable of improving planet detection and enabling the discovery of new worlds.

References

- Ardévol Martínez, F., Min, M., Kamp, I., & Palmer, P. I. 2022, *AA*, 662, A108
- Baron, D. 2019, *Machine Learning in Astronomy: a practical overview*, arXiv:1904.07248
- Basant, R., Luque, R., Bean, J. L., et al. 2025, Four sub-Earth planets orbiting Barnard’s Star from MAROON-X and ESPRESSO, arXiv:2503.08095
- Carter, J. A., Yee, J. C., Eastman, J., Gaudi, B. S., & Winn, J. N. 2008, *ApJ*, 689, 499
- Charbonneau, D., Brown, T. M., Latham, D. W., & Mayor, M. 2000, *The Astrophysical Journal*, 529, L45–L48
- Cochran, W. D., Hatzes, A. P., Endl, M., et al. 2002, in *AAS/Division for Planetary Sciences Meeting Abstracts*, Vol. 34, AAS/Division for Planetary Sciences Meeting Abstracts #34, 42.02
- de Beurs, Z. L., Vanderburg, A., Shallue, C. J., et al. 2022, *The Astronomical Journal*, 164, 49
- Desidera, S., Damasso, M., Gratton, R., et al. 2023, *A&A*, 675, A158
- Hellier, C., Anderson, D. R., Collier Cameron, A., et al. 2009, *Nature*, 460, 1098
- . 2017, *MNRAS*, 465, 3693
- Henry, G. W., Marcy, G. W., Butler, R. P., & Vogt, S. S. 2000, *ApJ*, 529, L41
- Iglesias Álvarez, S., Díez Alonso, E., Sánchez Rodríguez, M. L., et al. 2023, *Axioms*, 12, doi:10.3390/axioms12040348
- Kiefer, F. 2019, *Astronomy and Astrophysics*, 632, L9
- Latham, D. W., Mazeh, T., Stefanik, R. P., Mayor, M., & Burki, G. 1989, *Nature*, 339, 38

REFERENCES

- Mahabal, A., Sheth, K., Gieseke, F., et al. 2017, in 2017 IEEE Symposium Series on Computational Intelligence (SSCI) (IEEE), 1–8
- Mayor, M., & Queloz, D. 1995, *Nature*, 378, 355
- Morvan, M., Nikolaou, N., Yip, K. H., & Waldmann, I. 2022, Don’t Pay Attention to the Noise: Learning Self-supervised Representations of Light Curves with a Denoising Time Series Transformer, arXiv:2207.02777
- Ricker, G. R., Winn, J. N., Vanderspek, R., et al. 2014, *Journal of Astronomical Telescopes, Instruments, and Systems*, 1, 014003
- Shallue, C. J., & Vanderburg, A. 2018, *The Astronomical Journal*, 155, 94
- Smith, J. C., Stumpe, M. C., Van Cleve, J. E., et al. 2012, *PASP*, 124, 1000
- Smith, M. J., & Geach, J. E. 2023, *Royal Society Open Science*, 10, doi:10.1098/rsos.221454
- Struve, O. 1952, *The Observatory*, 72, 199
- van de Kamp, P. 1969, *AJ*, 74, 757
- Vasist, M., Rozet, F., Absil, O., et al. 2023, *Astronomy amp; Astrophysics*, 672, A147
- Wolszczan, A., & Frail, D. A. 1992, *Nature*, 355, 145
- Yu, L., Vanderburg, A., Huang, C., et al. 2019, *AJ*, 158, 25
- Zuckerman, B. 2014, Recognition of the First Observational Evidence of an Extrasolar Planetary System, arXiv:1410.2575